ONLINE MASTERS IN **DATA SCIENCE**

DSC258: NATURAL LANGUAGE PROCESSING

BAG OF WORDS REPRESENTATIONS

UC San Diego

COMPUTER SCIENCE & ENGINEERING

HALICIOĞLU DATA SCIENCE INSTITUTE

Bag-of-words Text Classifier

- What's text classification? How to evaluate a classifier?
- **Bag-of-words Representations**
- Building a linear classifier upon bag-of-words

Ignore word ordering but keep the "counts"

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	

Pre-processing Example

The following snippet is directly copied from lecture 0 with little revision

In [29]:
```
import nltk
nltk.download('punkt') # downloads you a model

nltk.download('stopwords') # <--- this is new
from nltk.corpus import stopwords
stop = set(stopwords.words('english'))
print(stop)
```

```
{'doing', "hadn't", 'her', 'these', 'should', 'was', "you're", 'nor', 'themselves', 'myself', 'with', 'have', 'be',
"aren't", "wouldn't", 'down', 'into', 'him', 't', 'itself', 'all', 'they', 'this', 'were', "doesn't", "shan't", 're',
'over', 'mustn', 'how', "mightn't", 'them', 'ma', 'hadn', 'until', 'you', 'before', 'don', 's', "shouldn't", 'yours',
'if', 'yourselves', "won't", 'it', 'not', 'some', 'same', 'what', 'when', "she's", 'between', 'mightn', 'or', 'most',
"weren't", 'm', 'hasn', 'ours', 'up', 'while', 'against', "didn't", 'had', 'because', 'few', "should've", 'will', 'ne
edn', 'weren', 'hers', 'once', "needn't", 'off', 'just', "don't", 'y', "isn't", 'doesn', 'after', 'shouldn', 'and',
'in', 'out', 'can', "you've", "you'd", 'from', 'those', "couldn't", 'during', 'did', 'any', 'where', 'through', 'havi
ng', 'wasn', 'other', 'am', 'about', 'both', 'me', 'couldn', 'such', 'is', 'the', 'whom', 'shan', 'your', "that'll",
'at', 'are', 'aren', 'yourself', 'now', 'wouldn', 'that', 'himself', 'to', 'we', 'has', 'been', 'his', 'for', 'ther
e', 'which', 'd', "hasn't", 'he', "wasn't", 'very', 'only', 'but', 'no', 'herself', 'being', 'll', 'who', 'than', 'do
es', 'then', 'so', 'further', 'each', 'above', 'an', 'as', 'a', 'she', 'our', "you'll", 'by', 'why', 'haven', 'ain',
'its', 'below', 'didn', 'isn', 'here', 'theirs', 'do', 've', 'own', 'more', 'o', 'again', 'on', "it's", "haven't", 'u
nder', "mustn't", 'my', 'their', 'ourselves', 'too', 'of', 'i', 'won'}
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     /Users/shangjingbo/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/shangjingbo/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [32]:
```
print('An' in stop) # <--- case sensitive in python
print('an' in stop)
print("'s" in stop)
```

```
False
True
False
```

Pre-processing Example

In [33]:
```
import nltk
nltk.download('punkt') # downloads you a model

nltk.download('stopwords') # <--- this is new
from nltk.corpus import stopwords
stop = set(stopwords.words('english'))

# print(stop)

from nltk.tokenize import word_tokenize
from nltk.tokenize import sent_tokenize
from nltk.stem import PorterStemmer

ps = PorterStemmer()

# return a list of tokens
def pre_processing_by_nltk(doc, stemming = True, need_sent = False):
    # step 1: get sentences
    sentences = sent_tokenize(doc)
    # step 2: get tokens
    tokens = []
    for sent in sentences:
        words = word_tokenize(sent)
        # step 3 (optional): stemming
        if stemming:
            words = [ps.stem(word) for word in words]
        if need_sent:
            tokens.append(words)
        else:
            tokens += words
    return [w.lower() for w in tokens if w.lower() not in stop]
```

Pre-processing Example

```
In [34]: df.review[:10].apply(pre_processing_by_nltk)
```

```
Out[34]: 0    [1974, ,, teenag, martha, moxley, (, maggi, gr...
         1    [ok, ..., ..., realli, like, kri, kristofferso...
         2    [*, *, *, spoiler, *, *, *, read, thi, ,, thin...
         3    [hi, peopl, seen, thi, wonder, movi, im, sure,...
         4    [recent, bought, dvd, ,, forget, much, hate, m...
         5    [leav, braik, put, good, show, ., final, zorak...
         6    [nathan, detroit, (, frank, sinatra, ), manag,...
         7    [understand, ``, crash, cours, '', right, cont...
         8    ['ve, impress, chavez, 's, stanc, globalis, so...
         9    [thi, movi, direct, renni, harlin, finnish, mi...
         Name: review, dtype: object
```

How to define weights?

- Weights should reflect the importance

- Choices
  - Binary
  - Term Frequency (TF)
  - TF + Inverse Document Frequency (TF-IDF)

Binary Bag-of-words Vectors

- This is a flexible framework
  - How to define term? → all words in the vocabulary
  - How to assign weights? → binary: present or not

- Vocabulary $V = \{v_1, v_2, \dots, v_n\}$

- Given a document $d = (w_1, w_2, \dots, w_m)$ ← these words are ordered

$$\boldsymbol{x}_{d,i} = \begin{cases} 1 & \exists j, s.t. v_i = w_j \\ 0 & otherwise \end{cases}$$

Example

- V = {news, about, presidential, campaign, food, …}
- D1: … news about presidential campaign …
- D2: … news about organic food campaign …
- D3: … news of presidential campaign …
- D4: … campaign … campaign … campaign … campaign … campaign …

- $\boldsymbol{x}_{D1} = (1, 1, 1, 1, 0, \dots)$
- $\boldsymbol{x}_{D2} = (1, 1, 0, 1, 1, \dots)$
- $\boldsymbol{x}_{D3} = (1, 0, 1, 1, 0, \dots)$
- $\boldsymbol{x}_{D4} = (0, 0, 0, 1, 0, \dots)$

From Binary to TF

- This is a flexible framework
  - How to define term? → all words in the vocabulary
  - How to assign weights? → frequency: how many times it appears

- Vocabulary $V = \{v_1, v_2, \dots, v_n\}$

- Given a document $d = (w_1, w_2, \dots, w_m)$ ← these words are ordered

$$\boldsymbol{x}_{d,i} = \left|\{j \mid w_j = v_i\}\right|$$

Example

- V = {news, about, presidential, campaign, food, …}
- D1: … news about presidential campaign …
- D2: … news about organic food campaign …
- D3: … news of presidential campaign …
- D4: … campaign … campaign … campaign … campaign … campaign …

- $\boldsymbol{x}_{D1} = (1, 1, 1, 1, 0, \dots)$
- $\boldsymbol{x}_{D2} = (1, 1, 0, 1, 1, \dots)$
- $\boldsymbol{x}_{D3} = (1, 0, 1, 1, 0, \dots)$
- $\boldsymbol{x}_{D4} = (0, 0, 0, 5, 0, \dots)$

More TF Variants

- Idea: A term is more important if it occurs more frequently in a document
- $f(t,d)$ is the raw frequency of the term t in the document d
- Popular Variants of TF
  - Raw TF: TF(t,d) = $f(t,d)$
  - Log TF: TF(t,d) = $\log(f(t,d)+1)$
  - Maximum Frequency Normalization: TF(t,d) = $0.5 + 0.5\times f(t,d)/$MaxFreq(d)
  - BM25 TF (normalized by document length): TF(t,d) = $\frac{(\mathrm{k}+1)\times f(t,d)}{f(t,d)+k(1-b+b\times\frac{|d|}{E[|d|]})}$

Normalization is important!

- “Repeated occurrences” are less informative than the “first occurrence”

- Generally penalize long document, but avoid over-penalizing (pivoted normalization)

- Two possible reasons of a long document:
  - A doc is long because it uses more words
  - A doc is long because it has more contents

Inverse Document Frequency

- Idea: A term is more discriminative/important if it occurs only in fewer documents

- $df_D(t)$ is the number of documents that contains term t in the corpus D
  - This D is expected to be a **huge** collection of documents
  - This D could be some other corpus

- Variants of IDF
  - Log IDF: IDF(t) = $1 + \log\left(\frac{|D|}{df_D(t)}\right)$
  - Smoothed IDF: IDF(t) = $\log\left(\frac{|D|+1}{df_D(t)+0.5}\right)$

Pre-compute IDF

In [55]:
```
from tqdm import tqdm

DF = defaultdict(float)
for doc in tqdm(df.review):
    tokens = pre_processing_by_nltk(doc)
    for token in set(tokens):
        DF[token] += 1
```

```
100%|██████████| 50000/50000 [02:35<00:00, 322.37it/s]
```

In [56]:
```
from math import log
IDF, vocab = dict(), dict()
for token in DF:
    if DF[token] < 50:
        # this becomes an unk
        pass
    else:
        vocab[token] = len(vocab)
        IDF[token] = log(1 + len(df.review) / DF[token])
print(len(DF), len(vocab))
```

```
131316 7807
```

In [57]:
```
IDF['<UNK>'] = 1
vocab['<UNK>'] = len(vocab)
```

TF-IDF Vectorizer

In [60]:
```
def tfidf_feature_extractor(doc, vocab, IDF):
    tokens = pre_processing_by_nltk(doc)
    for i, token in enumerate(tokens):
        if token not in vocab:
            tokens[i] = '<UNK>'
    TF = defaultdict(int)
    for token in tokens:
        TF[token] += 1
    x = [0] * len(vocab)
    for token in set(tokens):
        tfidf = log(TF[token] + 1) * IDF[token]
        token_id = vocab[token]
#         print(token, TF[token], IDF[token])
        x[token_id] = tfidf # this will be a dense matrix
    return x


x = tfidf_feature_extractor('I love this movie movie', vocab, IDF)
print(x)
# very sparse x! better change it later to sparse vector.
```

In [61]:
```
X = []
for doc in tqdm(df.review):
    X.append(tfidf_feature_extractor(doc, vocab, IDF))
y = list(df.sentiment)
```

```
100%|██████████| 50000/50000 [02:41<00:00, 309.04it/s]
```

TF-IDF Weighting

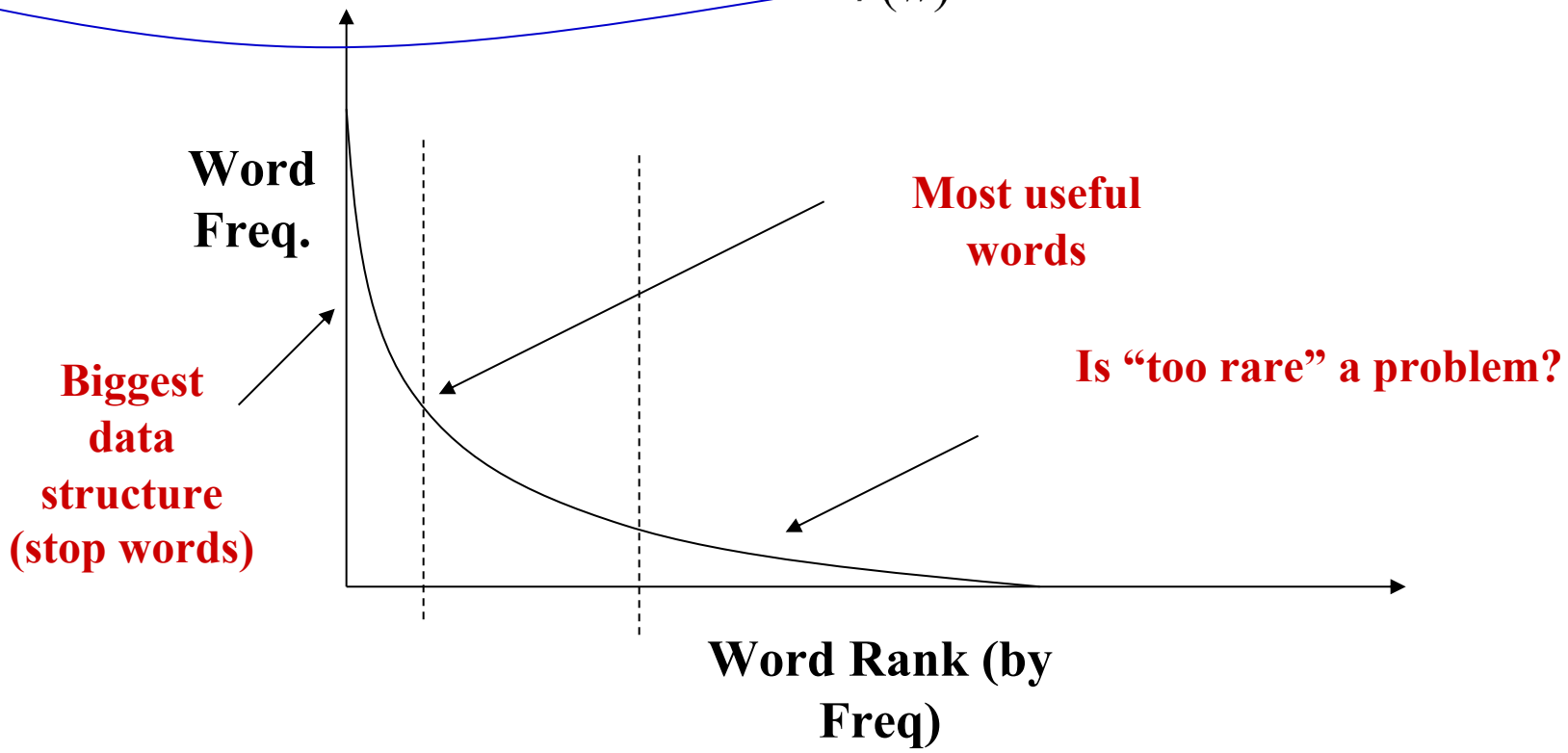
- Weight(t, d) = TF(t, d) * IDF(t)
  - Common in doc → high TF → high weight
  - Rare in collection → high IDF → high weight

- So, what kind of terms would have high weights in a document?

ZIPF'S LAW (Revisit)

- Rank * Frequency ≈ Constant

$$F(w) = \frac{C}{r(w)^{\alpha}} \qquad \alpha \approx 1, C \approx 0.1$$



Generalized Zipf's law: $F(w) = \frac{C}{[r(w)+B]^{\alpha}}$ **Applicable in many domains**

Bag-of-words Summary

- Pros
  - Empirically effective
  - Intuitive
  - Easy to implement
- Cons
  - Too many variants of TF-IDF
  - Ad-hoc parameter tuning
  - Assume term independence

- Tips
  - Sparse Vector/Matrix is your best friend in vector space modeling